# Week 8 — Project Proposal

| | |
|---|---|
| **Group Name / No.:** | PNP/1 |
| **Members:** | Kirk Quisagan, Ken Tamidles, Kimberly Luistro, Yvonne Mae Tapan, Venice Nadela, Nesley Cate Librea |
| **Repository URL:** | https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1.git |
| **Chosen Theme (circle one):** | **H** - Recipe Meal Planner |
| **API You Will Use:** | **TheMealDB** :https://www.themealdb.com/api.php (shared test key "1"). Additional option under consideration: a nutrition API from RapidAPI's Food APIs collection (https://rapidapi.com/collection/food-apis) |

## 1. One-Paragraph Pitch

Plate N Platter is a personal meal planner for individual users such as students and busy households. After registering and logging in, a user plans meals on a calendar where each meal is color-coded by Healthy Level and labeled Quick, Easy, or Hard to prepare. Users import real recipes from TheMealDB (including Filipino dishes) or create their own, including dishes built from other dishes, and the app adds up prep time, calories, and ingredients across them. A cooking queue, an undo stack, and a merged shopping list keep the user organized. Eating at least one Healthy meal a day builds a streak that earns reward badges, and a statistics page shows weekly charts and favorite dishes.

## 2. Entity-Relationship Sketch

(List your entities, their key attributes, and how they relate — text notation is fine, e.g., Member(id, name) --1:M--> Loan(id, memberId FK, bookId FK) <--M:1-- Book(id, title))

```
User(id PK, name, username UNIQUE, age, gender, passwordHash, salt)

Recipe(id PK, userId FK, name, mealType, healthyLevel, cuisine,
```

instructions,

prepTimeMinutes, servings, calories, protein, carbs, fat, sugar,

imageUrl, externalId)

Ingredient(id PK, name)

RecipeIngredient(recipeId FK, ingredientId FK, measure)

RecipeComponent(parentRecipeId FK, subRecipeId FK, quantity)

MealPlanEntry(id PK, userId FK, planDate, mealSlot, recipeId FK, eaten)

ShoppingListItem(id PK, userId FK, ingredientId FK, quantity, checked)

CookQueueEntry(id PK, userId FK, recipeId FK, addedAt)

Streak(userId PK/FK, currentStreak, bestStreak, cheatDaysAvailable, lastHealthyDay)

UserBadge(id PK, userId FK, badgeName, dateEarned)

**Stretch entity (built after the core features work)**

Favorite(userId FK, recipeId FK)

Relationships :

User --1:M--> Recipe

User --1:M--> MealPlanEntry --M:1--> Recipe

User --1:M--> ShoppingListItem --M:1--> Ingredient

User --1:M--> CookQueueEntry --M:1--> Recipe

User --1:1--> Streak

User --1:M--> UserBadge

Recipe --1:M--> RecipeIngredient <--M:1-- Ingredient (many-to-many via junction)

Recipe --1:M--> RecipeComponent --M:1--> Recipe

```
(self-reference: a recipe uses sub-recipes)

User --1:M--> Favorite --M:1--> Recipe(stretch)
```

Notes:

- **mealType** is Breakfast, MainCourse, or Dessert. **mealSlot** is Breakfast, Lunch, Dinner, or Snack. **healthyLevel** is Healthy, Moderate, or Indulgent.
- Difficulty (Quick / Easy / Hard) is not stored; it is derived from the recursive total of prep time across the recipe and its sub-recipes.
- Passwords are stored only as a salted hash, never as plain text.
- **externalId** holds TheMealDB's **idMeal** so the same recipe is not imported twice.
- TheMealDB provides no nutrition data, so calories, protein, carbs, fat, and sugar come from the user or, if we adopt the additional nutrition API, from that API (default values by Healthy Level otherwise).
- A streak day counts when at least one Healthy-level meal is marked eaten that day.
- Database plan: all tables live in local SQLite and are accessed through JDBC with **PreparedStatement**, as the spec requires. Cloudflare D1 (cloud SQLite) is our planned online sync of the same tables, pending instructor approval.

# 3. Feature Checklist

| Planned Feature | Requirement(s) It Satisfies |
|---|---|
| Register (name, username, age, gender, password, confirm password) and Login with salted password hashing and error messages | Database (JDBC PreparedStatement), Event-Driven GUI |
| Abstract Recipe with BreakfastRecipe, MainCourseRecipe, DessertRecipe, each with its own prep-time rule | Object-Oriented Design |
| Recursive methods that total prep time, calories, and ingredients across nested sub-recipes; total prep time then sets Quick / Easy / Hard (with protection against circular references) | Recursion |
| Cooking queue: "Add to Queue" on a recipe, first in, first out | Queue |

| | |
|---|---|
| Undo stack for adding, removing, and moving meals on the calendar | Stack |
| Shopping list that merges ingredients across planned meals (ingredient → total quantity) | Map |
| Weekly/monthly calendar grid with live date and time; meals colored by Healthy Level | Event-Driven GUI, Database |
| Today's Meals screen with Breakfast, Lunch, Dinner, and optional Snack, showing dish photos from the API | API Consumption, Event-Driven GUI |
| Meal Times screen: browse dishes, filter by Healthy Level and Low / Mid / High nutrition values, add to plan, and "Create your own meal" form | Event-Driven GUI (3+ controls), Database |
| Import recipes from TheMealDB by name, category, or ingredient, parsed from JSON into Recipe objects | API Consumption |
| API search and large CSV imports (recipe box and meal-plan history) run on a background Task with a loading indicator | Concurrency |
| Weekly statistics: meals by Healthy Level, most-planned category and cuisine, most-eaten dishes, average prep time, average daily calories | Data Mining |
| Statistics bar chart of meals per Healthy Level (weekly summary) | Data Visualization |
| Daily streak: eating at least one Healthy-level meal each day (marked eaten) continues the streak; milestones award badges (Full Week Planner, Recipe Collector, Perfect Week, Healthy Meal Planned) and cheat-day rewards | Data Mining (consecutive-day analysis of meal-plan records), Event-Driven GUI |
| 6+ JUnit 5 tests: password hashing, recursion (plain recipe, one sub-recipe, 3-level nest, empty recipe), queue order, undo stack, shopping-list merge, streak counting | Automated Testing |
| Filipino Dishes category: filter TheMealDB by cuisine (Filipino), supplemented by our own Filipino recipe CSV | API Consumption |

| | |
|---|---|
| if the API has few | |
| Optional cloud sync of the SQLite tables to Cloudflare D1 (needs instructor approval) | Database (extension) |
| Look up calories and nutrients for imported dishes from a nutrition API (additional option) and cache results in SQLite; used to help set Healthy Level | API Consumption, Database |

## 4. Task Split

| Member | Primary Responsibility (this milestone) |
|---|---|
| Kirk Quisagan | SQLite schema and JDBC data-access classes |
| Ken Tamidles | TheMealDB API client and JSON parsing (plus the nutrition API if adopted) |
| Kimberly Luistro | OOD classes (Recipe hierarchy, MealPlanEntry), cooking queue, and undo stack |
| Yvonne Mae Tapan | JavaFX login/register screens and Home layout |
| Venice Nadela | Calendar grid and Meal Times screen |
| Nesley Cate Librea | Recursion (prep time and difficulty), JUnit tests, statistics, streak tracking, and chart |

## 5. Public API You Plan to Use

TheMealDB (https://www.themealdb.com/api.php): a free recipe database that returns JSON with meal names, categories, regions, instructions, photo URLs, and up to 20 ingredient/measure pairs per meal. We will use the search, filter-by-ingredient or category, and lookup-by-ID endpoints. We plan to use its cuisine filter for the Filipino Dishes category and are still checking how many Filipino dishes it offers. More API are still being considered for Filipino options for finalization.

Additional option under consideration: a nutrition API from RapidAPI's Food APIs collection. TheMealDB gives recipes but no nutrition data, so we are considering one of the nutrition APIs listed there (Spoonacular Recipe-Food-Nutrition, Edamam Nutrition Analysis, or CalorieNinjas) to supply calories, protein, carbs, fat, and sugar for imported dishes and to help set their Healthy Level. These APIs need a free RapidAPI account and key and usually have request limits, so we will check the limits before choosing and cache every result in SQLite. TheMealDB remains our primary API.

# 6. Risks / Open Questions

- TheMealDB has no calories, nutrition, or prep-time data. We are considering a nutrition API from RapidAPI's Food APIs collection (Spoonacular, Edamam, or CalorieNinjas) to supply calories and nutrients for imported dishes, with user entry as the fallback. Is a second API acceptable?
- TheMealDB categories (Beef, Chicken, etc.) differ from our Breakfast / MainCourse / Dessert, so we need a mapping rule. We also need to confirm how many Filipino dishes the API actually offers; if few, we will add our own.
- Sub-recipes are not provided by the API. Users will link them in the app, and we must prevent circular references (A uses B, B uses A).
- Are Quick / Easy / Hard thresholds based on the recursive total prep time acceptable? The same recursion also totals calories and ingredients.
- Ingredient measures are free text ("1 cup", "a pinch"), which makes merging quantities in the shopping list tricky.
- Login accounts are stored in the local SQLite database. Passwords are hashed with a salt using Java's built-in libraries
- The shared test key "1" may have usage limits, so we will cache fetched recipes in SQLite.
- Database: the spec requires SQLite tables accessed via JDBC with PreparedStatement. Cloudflare D1 is SQLite-compatible but is reached over the internet (a community JDBC driver exists, or Cloudflare's REST API / a Worker). We propose local SQLite via JDBC as the main database and D1 as an optional cloud sync. Do we need approval to use D1, and is an API token inside a desktop app acceptable?
- Streak rule: a day counts when at least one Healthy-level meal is marked eaten. We still need to decide how cheat days protect a streak.
- RapidAPI keys must never be committed to the shared repository. We will keep the key in a local config file excluded by .gitignore, and check each API's free-plan request limits before choosing.

# Appendix: GUI Overview (from our UI visualization plan)

| Screen | What it shows |
|---|---|
| **Welcome / Login / Create Account** | Tagline, username and password login, registration form with confirm-password check |
| **Home** | "Plan my meals" entry, today's food choices, category shortcuts (Healthy Meals, Pasta, Filipino Dishes, Breakfast, Quick Meals, Dessert), side menu: Home, Calendar, Meal Times, Shopping List, Stats, Streak |
| **Calendar** | Week/month grid, today highlighted, daily calorie total, Healthy / Moderate / Indulgent color legend, badge-unlocked notice |
| **Today's Meals** | Breakfast, Lunch, Dinner, and optional Snack with dish photos and "View Details" |
| **Meal Times** | Category tabs, Low / Mid / High filters (calories, protein, carbohydrates, fat, sugar), dish cards, "Create your own meal" |
| **Recipe Details** | Ingredients and procedure with buttons: Add to Meal, Add to Queue, Add to Favorites, Add to Shopping List |
| **Shopping List** | Ingredients to buy with "Mark as complete" and "Delete" |
| **Streak** | Current streak (a day counts when a Healthy meal is eaten), next milestone, cheat days available, achievements and badges |
| **Statistics** | Weekly calories and average per day, meals by Healthy Level (bar chart), favorite dish, top 3 dishes, most-planned category, average prep time |